

Position: Full Stack Developer Intern (MERN)

Duration: 48 Hours

Deadline: 21 July, 2026 by 5 PM IST

Objective

Build "DevDrops" a minimalist developer productivity tool where users can capture, organize, and recall "knowledge drops" (quick notes, code snippets, command references, and links) with smart categorization and a unique "spaced repetition" recall system.

Core Requirements

1. Backend (Node.js + Express + MongoDB)

Authentication

- JWT-based authentication (register, login, logout)
- Protected routes for all operations
- Password hashing with bcrypt
- Session tracking (store last login IP and timestamp)

Database Design

Design MongoDB schemas for:

User

- username (unique, min 3 chars)
- email (unique, valid format)
- password (hashed, min 6 chars)
- lastLogin (Date)
- lastLoginIP (String)
- preferences: {
 - theme: (light/dark),
 - defaultVisibility: (public/private),
 - recallInterval: (Number, default: 24) // hours}
- createdAt

Drop (main entity)

- title (String, required, max 100 chars)
- content (String, required) // Can contain markdown
- type: (code, command, link, note) // Enum
- language: (String, optional, for code type)
- tags: [String] // Array, max 5 tags
- visibility: (public/private)
- createdBy: (ref: User)
- isFavorite: Boolean
- recallCount: Number // How many times recalled
- lastRecalled: Date
- nextRecallDate: Date // Calculated using Fibonacci sequence
- relatedDrops: [ref: Drop] // Self-referencing for connections
- createdAt
- updatedAt

Collection (folder alternative)

- name: (String, required, unique per user)
- description: (String, max 200 chars)
- createdBy: (ref: User)
- drops: [ref: Drop]
- color: (String, hex code)
- isShared: Boolean
- shareToken: String (optional, for public sharing)
- createdAt

RecallHistory

- dropId: (ref: Drop)
- userId: (ref: User)
- recalledAt: Date
- recallType: (manual, scheduled)
- confidence: (Number, 1-5) // Self-reported recall confidence

API Endpoints

Auth Routes

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login, returns JWT + user data
POST	/api/auth/logout	Logout
GET	/api/auth/me	Get current user profile
PUT	/api/auth/preferences	Update user preferences

Drop Routes (all protected)

Method	Endpoint	Description
GET	/api/drops	Get all drops (filter by type, tag, visibility) - paginated (8 per page)
GET	/api/drops/recall	Get drops due for recall (based on nextRecallDate)
GET	/api/drops/related/:id	Get related drops for a specific drop
POST	/api/drops	Create new drop (auto-calculate nextRecallDate)
POST	/api/drops/:id/recall	Mark drop as recalled (update recallCount, nextRecallDate using Fibonacci)
POST	/api/drops/:id/relate	Add related drop relationship (both ways)
PUT	/api/drops/:id	Update drop (owner only)
DELETE	/api/drops/:id	Delete drop (owner only)
GET	/api/drops/stats	Get recall statistics (streak, mastered drops, pending)

Collection Routes (all protected)

Method	Endpoint	Description
GET	/api/collections	Get all collections for user
POST	/api/collections	Create collection
PUT	/api/collections/:id	Update collection
POST	/api/collections/:id/drops	Add drop to collection
DELETE	/api/collections/:id/drops/:dropId	Remove drop from collection
DELETE	/api/collections/:id	Delete collection (drops remain)
GET	/api/collections/:id/share	Generate shareable link (public view)

Public Routes (no auth required)

Method	Endpoint	Description
GET	/api/public/share/:token	View shared collection (read-only)

| GET | ``/api/public/explore`` | Browse public drops (paginated) |

Middleware Required

- Authentication middleware (verify JWT)
- Ownership middleware (check drop/collection ownership)
- Request validation (Joi - strict validation)
- Rate limiting:
 - Auth routes: 5 requests per minute
 - Public endpoints: 20 requests per minute
 - Recall endpoints: 30 requests per minute
- Global error handling middleware
- Request logging middleware (log all requests with timestamp, method, path, IP)

Business Logic (Critical - Must Implement)

Spaced Repetition Algorithm (Fibonacci-based):

// When a drop is created:

`nextRecallDate = createdAt + 1 hour`

// When recalled successfully:

`recallCount++`

`nextRecallDate = currentDate + (Fibonacci(recallCount + 1) * 1 hour)`

// Fibonacci sequence: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...

// So recall intervals: 1h, 1h, 2h, 3h, 5h, 8h, 13h, 21h, 34h, 55h...

Auto-categorization:

- Detect drop type from content:
 - If content contains ````code block```` or ``function|class|const|let|var`` → type: "code"
 - If content starts with ``$`` or ``>`` or contains ``sudo|apt|npm|yarn|pip`` → type: "command"
 - If content contains ``http://`` or ``https://`` → type: "link"
 - Otherwise → type: "note"
- Automatically extract tags from content (max 5, use a simple keyword extraction)

Related Drops Logic:

- When creating a drop, search existing drops with similar tags or keywords in content
- Suggest related drops (return array of IDs)
- Allow manual linking via ``/api/drops/:id/relate``

Recall Statistics:

- Track: total drops, mastered (recalled 5+ times), pending (due for recall), current streak
- Streak: consecutive days with at least one recall

2. Frontend (React.js + Redux Toolkit + RTK Query)

Pages Required

Authentication Pages

- Login page with email/password + "Remember me" (localStorage)
- Register page with username, email, password, confirm password
- Password strength indicator (real-time)
- Redirect to dashboard after login

Dashboard (Main Hub)

- Welcome message with username
- Recall Queue - Cards showing drops due for recall (with count badge)
- Quick recall action (one-click "I remember this" or "Show me")
- Stats cards:
 - Total Drops
 - Mastered (recalled ≥ 5 times)
 - Pending (due for recall)
 - Current Streak (days)
- Recent drops (last 5 created/updated)
- Quick create drop button (floating action)

Recall Mode (Interactive Page)

- Display one drop at a time from the recall queue
- Initially show only title + type badge (hide content)
- User clicks "Reveal" to show content
- After revealing: "I Remembered" or "Need to Review" buttons
- Track confidence rating (1-5 stars) after reveal
- Show progress: "3 of 7 drops recalled today"
- Keyboard shortcuts: Space (reveal), R (remembered), N (need review)

Drops Explorer

- Search bar (by title, content, tags)
- Filter dropdowns: Type, Language, Visibility (public/private)
- Sort options: Newest, Most Recalled, Alphabetical
- Pagination (8 per page, load more button)
- List view with:
 - Title, type badge, first 50 chars of content preview
 - Tags (with color-coded chips)
 - Recall count, last recalled date
 - Favorite toggle star
- Click drop to view full content (opens modal or detail page)

Drop Detail Page

- Full content display with markdown rendering
- Type badge and language (if code)
- Tags display
- "Recall this drop" button (manual recall)
- "Add Related Drop" button with suggestions
- Related drops section (clickable cards)
- Statistics: total recalls, next recall date, created date
- Edit/Delete buttons (owner only)
- Add to collection dropdown

My Drops Page

- Tabs: All / Favorite / Due for Recall / By Type
- Create new drop button
- Grid/list view toggle
- Bulk actions: select multiple drops → delete, add to collection, change visibility
- Filter by collection
- Search within my drops

Collections Page

- Grid of collections with color-coded cards
- Collection stats: drop count, share status
- Create/rename/delete collections
- Click collection to view all drops inside
- Share button → generates public link (copyable)
- Public view page (lightweight, no auth, read-only)

Profile Page

- User info (username, email, last login)
- Statistics: total drops, collections, mastered drops, streak
- Edit preferences:
 - Theme toggle (dark/light)
 - Default visibility (public/private)
 - Recall interval customization (hours)
- Export all data (JSON download)

State Management

- RTK Query: All API calls with caching
- Redux Slice: Auth state + UI state (theme, current filters, recall queue)
- Optimistic updates: For favorite toggle, recall marking, delete
- Cache invalidation: On drop create/update/delete, recall, relate
- Persist: Theme preference, recall state, "Remember me" token

Special UI Requirements

- Markdown rendering (react-markdown)
- Code syntax highlighting (prism.js with language support)
- Toast notifications (react-hot-toast)
- Confetti effect on mastering a drop (5th recall)
- Keyboard shortcuts throughout app:
 - `Ctrl/Cmd + K`: Quick search/command palette
 - `Ctrl/Cmd + N`: New drop
 - `Esc`: Close modals
 - `R`: Recall mode (when available)
- Progress indicators for recall queue
- Loading skeletons for all list views
- Empty states with fun illustrations (use emoji-based art)

3. Technical Constraints

Backend Constraints:

1. No ORM/ODM - Use native MongoDB driver only (no Mongoose)
2. Custom validation - Write custom validation functions, no validation libraries (except Joi for schemas)
3. Manual aggregation pipelines - Must use `\$lookup`, `\$unwind`, `\$group` directly

4. No `async/await` in controllers - Use callbacks or `.then().catch()` (to prevent common AI patterns)
5. Custom Logger - Build your own logging utility with log levels (error, warn, info, debug)
6. No external libraries for spaced repetition (implement algorithm from scratch)

Frontend Constraints:

1. No UI component library - Use Tailwind CSS only (no DaisyUI, ShadCN, MUI, etc.)
2. No pre-built markdown renderers - Build a simple markdown parser (support: headers, bold, italic, code blocks, links)
3. No form libraries - Build custom form handling with useReducer (no Formik, React Hook Form)
4. No animation libraries - Write custom CSS animations (fade, slide, bounce)
5. Write custom hooks for:
 - useLocalStorage
 - useKeyboardShortcut
 - useDebounce
 - useRecallQueue
6. Create a custom command palette (Ctrl+K) from scratch

Database Constraints:

1. Create indexes manually (write MongoDB shell commands in /server/scripts/indexes.js)
2. Write a seed script that generates realistic test data with relationships
3. Implement transactions for: deleting a drop with recall history, bulk operations
4. Write a MongoDB view for drop statistics (aggregation)

Deployment Constraints:

1. Write your own Dockerfile for both frontend and backend
2. Set up nginx as reverse proxy (can use local dev setup)
3. Environment variables must be validated on startup (throw error if missing)

4. Deployment

Services	Platform
Frontend	Vercel
Backend	Render/Railway

Database	MongoDB Atlas
Container	Docker (optional but recommended)

5. Submission Checklist

```

devdrops/
├── client/
│   ├── src/
│   │   ├── components/
│   │   │   ├── common/
│   │   │   │   ├── Button.jsx
│   │   │   │   ├── Input.jsx
│   │   │   │   ├── Card.jsx
│   │   │   │   ├── Modal.jsx
│   │   │   │   ├── Toast.jsx (custom)
│   │   │   │   └── LoadingSpinner.jsx
│   │   │   ├── dashboard/
│   │   │   ├── drops/
│   │   │   ├── collections/
│   │   │   └── profile/
│   │   ├── pages/
│   │   ├── store/
│   │   │   ├── slices/
│   │   │   └── api/
│   │   ├── hooks/
│   │   │   ├── useLocalStorage.js
│   │   │   ├── useKeyboardShortcut.js
│   │   │   └── useRecallQueue.js
│   │   ├── utils/
│   │   │   ├── markdownParser.js (custom)
│   │   │   ├── fibonacci.js
│   │   │   ├── tagExtractor.js
│   │   │   └── validators.js
│   │   ├── styles/
│   │   │   └── index.css (Tailwind + custom)
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── public/
│   ├── Dockerfile
│   ├── nginx.conf
│   ├── package.json
│   └── postcss.config.js

```

```

├── tailwind.config.js
├── .env.example
├── server/
│   ├── src/
│   │   ├── controllers/
│   │   │   ├── authController.js
│   │   │   ├── dropController.js
│   │   │   ├── collectionController.js
│   │   │   └── publicController.js
│   │   ├── models/
│   │   │   ├── userModel.js (no Mongoose)
│   │   │   ├── dropModel.js
│   │   │   ├── collectionModel.js
│   │   │   └── recallHistoryModel.js
│   │   ├── routes/
│   │   ├── middleware/
│   │   │   ├── auth.js
│   │   │   ├── ownership.js
│   │   │   ├── rateLimiter.js (custom)
│   │   │   ├── logger.js (custom)
│   │   │   └── errorHandler.js
│   │   ├── services/
│   │   │   ├── recallService.js (spaced repetition logic)
│   │   │   ├── tagService.js
│   │   │   └── searchService.js
│   │   ├── db/
│   │   │   ├── connection.js (native driver)
│   │   │   └── indexes.js
│   │   ├── scripts/
│   │   │   └── seed.js
│   │   ├── app.js
│   │   └── server.js
│   ├── Dockerfile
│   ├── package.json
│   └── .env.example
├── docker-compose.yml
├── README.md
└── .gitignore

```

6. README.md Requirements

Must include:

- Project Name & Tagline
- Live Demo Links (frontend + API health endpoint)
- Tech Stack (with versions)
- Features (checklist with all implemented features)
- Screenshots (at least 5: Dashboard, Recall Mode, Explorer, Detail Page, Collections)
- Setup Instructions (local development with Docker and without)
- Environment Variables (detailed table)
- API Documentation (all endpoints with examples)
- Database Schema (ERD or collection definitions)
- Algorithm Explanation: Spaced repetition implementation
- Challenges & Solutions: Minimum 3 challenges and how you solved them
- Performance Optimizations: What you did to make it fast
- Future Improvements: At least 5

7. Submission

Email: hr@codearies.com and info@codearies.com

Subject: DevDrops Submission - [Your Name]

Include:

- GitHub repo link (private with access granted)
- Frontend URL
- Backend health check URL
- 2-minute Loom/Video recording explaining your approach (optional but recommended)

⚠ Important Notes

1. No third-party services except MongoDB Atlas, Vercel, Render/Railway
2. All code must be original - no boilerplate templates
3. Git commits should show incremental development (not just one big commit)
4. README must be comprehensive - acts as your documentation
5. Mobile-first approach - build for mobile then scale up
6. Accessibility - proper ARIA labels, keyboard navigation, focus management

Good luck! You'll need it.